

1. Given two binary search trees T_A and T_B , please make these two binary search tree united as one binary search tree. Your solution can not be simply deleting each node in one tree and inserting them into another one sequentially. Please also show the time and space complexity of your approach.

Function Merge (treeA, treeB):

array A = []

array B = []

Convert To Inorder Array (arrayA, treeA)

Convert To Inorder Array (arrayB, treeB)

result = []

i = 0

j = 0

while i < arrayA.length && j < arrayB.length :

If arrayA[i] < arrayB[j] :

result.append (arrayA[i])

i += 1

ELSE :

result.append (arrayB[j])

j += 1

result.extend (array A)

result.extend (array B)

RETURN Convert To BST (result, 0, result.length)

Function ConvertTo BST (arr, i, j):

mid = (i + j) / 2

node = Node (arr[mid])

node.left = ConvertTo BST (arr, i, mid-1)

node.right = ConvertTo BST (arr, mid+1, j)

Return node

Function Convert To Inorder Array (arr, n)

If n is None:

Return

Convert To Inorder Array (arr, n.left)

arr.append (n.value)

Convert To Inorder Array (arr, n.right)

設 T_A 有 N 個節點, T_B 有 M 個節點

	時間複雜度	空間複雜度
Convert To BST	$O(M+N)$	--
Convert To Inorder Array	$O(M+N)$	--
Merge	$O(M+N)$	$O(M+N)$

2. Consider an implementation of heap H by means of a linked structure.

- (a) Suppose that we use a reference to the last node in the structure. Please show how to update the reference to the last node after the operations `insert()` and `removeMin()` $O(\log n)$ time, where $H.size = n$ is the current number of nodes of H . Be sure and handle all possible cases.
- (b) Design an $O(\log n)$ -time algorithm for finding the last node of H with $H.size = n$ nodes in the implementation of a heap by means of a linked structure that does not keep a reference to the last node.

(a) insert: (1) 樹是空的: 直接讓 last 指向新節點

(2) last 是左側子節點: 將新節點插入父節點的右邊,

且 $last = last.parent.right$

(3) last 是右子節點: 1. 一路往上爬, 直到自己變成左子節點

2. 爬到 $current.parent.left$

3. 往左下走到底, $last = current$

▲ 如果爬到 root 自己都不是左子節點, 就從 root 往左下走到底

removeMin: (1) last 是 root: $last = None$

(2) last 是右子節點: $last = last.parent.left$

(3) last 是左子節點: 1. 一路往上爬, 直到自己變成右子節點

2. 爬到 $current.parent.left$

3. 往右下走到底, $last = current$

▲ 如果爬到 root 自己都不是右子節點, 就從 root 往右下走到底

(b) Function FindLast (root, size):

index = 1

while index * 2 ≤ size: index *= 2

index >>= 1

while index != 0:

if size & index:

root = root.right

else

root = root.left

index >>= 1

return root

3. Let S be a set of n points in the plane with distinct integer x - and y -coordinates. Let T be a complete binary tree storing the points from S at its external nodes, such that the points are ordered left-to-right by increasing x -coordinates. For each node v in T , let $S(v)$ denote the subset of S consisting of points stored in the subtree rooted at v . For the root r of T , define $top(r)$ to be the point in $S = S(r)$ with maximum y -coordinate. For every other node v , define $top(v)$ to be the point in S with highest y -coordinate in $S(v)$ that is not also the highest y -coordinate in $S(u)$, where u is the parent of v in T (if such a point exists). Such labeling turns T into a *priority search tree*. Describe a linear-time algorithm for turning T into a priority search tree.

Function ChooseMax (node):

if node.isLeaf():

node.max = node.point

return

ChooseMax (node.left)

ChooseMax (node.right)

if node.left.max.y > node.right.max.y :

node.winner = node.left.winner

node.source = "LEFT"

else:

node.winner = node.right.winner

node.source = "RIGHT"

Function Main():

ChooseMax (node)

AssignMax (node, True)

Function AssignMax (node, max_taken):

If node.isLeaf():

return

If max_taken =

If node.left.max.y > node.right.max.y :

top (node) = node.left.max

AssignMax (node.left, True)

AssignMax (node.right, False)

Else:

top (node) = node.right.max

AssignMax (node.left, False)

AssignMax (node.right, True)

Else:

top (node) = node.max

if node.source == "LEFT" :

AssignMax (node.left, True)

AssignMax (node.right, False)

ELSE:

AssignMax (node.left, False)

AssignMax (node.right, True)